

LAPORAN TUGAS 2 (1 - 4)

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

JHODI ANDIKA (2410651050)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Dalam era komputasi modern yang didominasi oleh pemrosesan paralel dan sistem terdistribusi, pemahaman mendalam mengenai manajemen proses dan komunikasi antar proses (Interprocess Communication/IPC) telah menjadi kebutuhan pokok dalam perancangan arsitektur perangkat lunak. Isolasi proses yang menjadi pilar keamanan dan stabilitas sistem justru menciptakan tantangan tersendiri dalam hal koordinasi dan pertukaran data antar entitas eksekusi yang independen. Eksperimen ini dirancang untuk mengimplementasikan dua paradigma IPC utama melalui dua skenario berbeda: sistem manajemen proses yang memanfaatkan pola fork-exec-wait dan aplikasi komunikasi real-time berbasis shared memory dengan sinkronisasi semaphore.

Implementasi teknik IPC melalui shared memory dan message passing pipes menjadi krusial untuk memahami bagaimana proses-proses yang terisolasi dapat berkolaborasi secara efektif dalam menyelesaikan masalah komputasi yang kompleks. Sementara itu, mekanisme sinkronisasi seperti semaphore dan mutex berperan sebagai penjaga konsistensi data dalam lingkungan yang melibatkan akses bersamaan. Penguasaan konsep process lifecycle, alokasi sumber daya, dan pencegahan race condition melalui mekanisme IPC yang tepat menjadi kompetensi strategis yang tidak terelakkan dalam pengembangan sistem terdistribusi dan aplikasi multithreading yang efisien, khususnya dalam landscape komputasi paralel yang terus berkembang pesat.

Eksperimen ini tidak hanya bertujuan untuk memverifikasi teori-teori fundamental sistem operasi, tetapi juga memberikan wawasan praktis tentang pertimbangan desain arsitektural dalam memilih mekanisme IPC yang optimal berdasarkan karakteristik beban kerja dan kebutuhan aplikasi.

1.2 RUMUSAN MASALAH

1. Bagaimana implementasi manajemen proses dalam sistem operasi menggunakan teknik fork-exec-wait pattern untuk membuat multiple child processes yang melakukan tugas komputasi berbeda secara paralel?
2. Bagaimana mekanisme Interprocess Communication (IPC) dengan shared memory dan semaphore diimplementasikan untuk membangun aplikasi chat real-time yang aman dari race condition?
3. Bagaimana efektivitas sinkronisasi menggunakan semaphore dalam menjamin konsistensi data dan mencegah konflik akses pada shared memory ketika multiple processes mengakses sumber daya yang sama secara bersamaan?

1.3 TUJUAN

1. Menganalisis dan mempraktikkan manajemen proses hierarchical melalui pembuatan parent-child processes dengan tugas berbeda (sorting, searching, calculation) serta mengukur performa eksekusi menggunakan time measurement.
2. Mengimplementasikan aplikasi chat berbasis IPC dengan memanfaatkan shared memory sebagai media penyimpanan pesan terpusat dan semaphore binary sebagai mekanisme sinkronisasi critical section.
3. Mengevaluasi efektivitas mekanisme sinkronisasi dalam menjamin mutual exclusion dan data consistency pada lingkungan multiprocessing melalui testing real-time chat application dengan multiple concurrent users.

BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

2.1.1 Sistem Manajemen Proses

Program ini merupakan simulasi nyata dari konsep multiprocessing pada sistem operasi, di mana beberapa tugas dieksekusi secara bersamaan untuk meningkatkan efisiensi. Inti dari program ini adalah pembuatan tiga proses anak yang independen menggunakan pola fork-exec. Setiap proses anak diberi tanggung jawab komputasi yang spesifik: melakukan pengurutan data, melakukan pencarian dalam array, serta menghitung nilai rata-rata, maksimum, dan minimum.

Proses induk bertindak sebagai pengendali pusat yang tidak hanya meluncurkan tetapi juga mengawasi ketiga proses anak ini. Untuk memfasilitasi pertukaran data, proses induk menyiapkan beberapa pipe sebagai saluran komunikasi satu arah sebelum proses anak dibuat. Hal ini memungkinkan setiap anak untuk mengirimkan hasil perhitungan dan data kinerjanya (seperti Process ID dan lama waktu eksekusi yang diukur dengan ketelitian mikrodetik) kembali ke induk. Mekanisme `waitpid()` digunakan oleh proses induk untuk secara aktif menunggu proses anak menyelesaikan tugasnya. Ini memastikan sinkronisasi, mencegah kondisi race, dan memungkinkan induk untuk mengumpulkan status keluar dari setiap anak. Setelah semua hasil terkumpul, proses induk bertugas mengkonsolidasi dan menampilkan informasi tersebut dalam bentuk yang terstruktur, lengkap dengan total waktu yang dihabiskan seluruh sistem untuk menyelesaikan semua tugas.

```
dany@LAPTOP-GRJL2V1D:~$ cd jhodiandika
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ touch management_process.c
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ nano management_process.c
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ gcc -o management_process management_process.c
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ ./management_process
=== SISTEM MANAJEMEN PROSES SEDERHANA ===
Parent Process PID: 482

Process 483 (Sorting): Mengurutkan array...
Process 483 (Sorting): Array telah terurut dengan benar
Process 484 (Searching): Mencari nilai dalam array...
Process 484 (Searching): Nilai 42 ditemukan pada index 39

=== PARENT PROCESS MENUNGGU CHILD SELESAI ===
Process 485 (Calculation): Menghitung statistik...
Process 485 (Calculation): Sum=47684, Min=11, Max=996, Average=476.84

=== HASIL EKSEKUSI SEMUA PROSES ===


| PID | Task              | Time (s) | Status  |
|-----|-------------------|----------|---------|
| 483 | Sorting Array     | 0.000121 | Success |
| 484 | Searching Value   | 0.000049 | Success |
| 485 | Calculation Stats | 0.000091 | Success |



=== STATISTIK KESELURUHAN ===
Total processes: 3
Total execution time: 0.000261 seconds
Parent Process PID: 482
Semua child processes telah selesai dieksekusi!
dany@LAPTOP-GRJL2V1D:~/jhodiandika$
```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

A. PROSES KOMPILASI:

- Preprocessing: Dilakukan ekspansi terhadap semua direktif preprocessor seperti #include dan makro
- Kompilasi: GCC menerjemahkan kode C menjadi bahasa assembly yang spesifik terhadap arsitektur prosesor
- Assembly: Kode assembly diubah menjadi machine code dalam bentuk object file
- Linking: Digabungkan dengan library sistem yang diperlukan untuk membentuk executable yang lengkap
- Optimasi: Compiler melakukan berbagai optimasi level tinggi untuk meningkatkan kinerja kode

B. INISIALISASI SISTEM:

- Loading: Sistem operasi memuat binary executable ke dalam main memory
- Process Control Block: Kernel membuat PCB untuk mengelola state dan konteks proses
- Memory Mapping: Dilakukan alokasi memory space untuk kode, data, heap, dan stack segments
- Resource Allocation: Sistem mengalokasikan file descriptors dan resources lainnya
- Context Setup: Inisialisasi register CPU dan program counter untuk eksekusi

C. MANAJEMEN PROSES:

- Fork Mechanism: System call fork() membuat duplikasi proses parent yang identik
- Process Hierarchy: Terbentuk struktur tree dengan parent sebagai root process
- Address Space: Setiap child mendapatkan copy-on-write memory space yang terisolasi
- PID Assignment: Kernel memberikan Process ID unik untuk setiap child process
- Scheduling: Process scheduler mengatur alokasi CPU time untuk setiap proses

D. DISTRIBUSI TUGAS:

- Parallel Processing: Tiga child processes dieksekusi secara konkuren:
- Process A: Mengimplementasikan sorting algorithm dengan optimasi
- Process B: Menangani pattern matching dan data retrieval
- Process C: Melakukan numerical computation dan statistical analysis
- Workload Balancing: Beban komputasi didistribusikan secara merata
- Specialized Execution: Setiap proses fokus pada tugas spesifik untuk efisiensi

E. SISTEM KOMUNIKASI:

- Pipe Establishment: Dibuat multiple unidirectional pipes untuk komunikasi data
- Data Serialization: Hasil komputasi dikemas dalam structured format untuk transfer
- IPC Protocol: Implementasi protokol komunikasi antar proses yang reliable
- Buffer Management: Pengelolaan memory buffers untuk membaca dan menulis data
- Flow Control: Mekanisme untuk mencegah pipe overflow dan data loss

F. SINKRONISASI:

- Process Barrier: `waitpid()` berfungsi sebagai synchronization barrier
- Execution Ordering: Memastikan urutan eksekusi yang deterministik
- Race Condition Prevention: Mengeliminasi konflik akses resource bersama
- State Monitoring: Parent memantau status eksekusi setiap child process
- Timeout Handling: Mekanisme untuk menangani proses yang hang atau stalled

G. MANAJEMEN WAKTU:

- High-Resolution Timing: Pengukuran waktu dengan presisi mikrodetik
- Performance Profiling: Pencatatan execution time untuk setiap task
- Benchmarking: Perbandingan kinerja antar proses yang berbeda
- Total Runtime Calculation: Akumulasi waktu eksekusi keseluruhan sistem
- Overhead Measurement: Termasuk waktu yang dihabiskan untuk context switching

H. KONSOLIDASI HASIL:

- Data Aggregation: Pengumpulan dan penggabungan hasil dari semua processes
- Result Validation: Pemeriksaan validitas dan konsistensi data
- Format Transformation: Konversi data ke format output yang terstruktur
- Summary Generation: Pembuatan laporan eksekusi yang komprehensif
- Error Reporting: Pelaporan status error dan exception handling

I. CLEANUP & TERMINASI:

- Resource Reclamation: Pelepasan semua allocated resources kembali ke sistem
- File Descriptor Cleanup: Penutupan semua pipes dan open files
- Memory Deallocation: Pembebasan dynamic memory dan cleanup stacks
- Process Termination: Exit graceful dengan return code yang appropriate
- Zombie Prevention: Memastikan tidak ada proses yang tertinggal dalam state zombie

J. OPTIMISASI SISTEM:

- Parallelism Benefit: Peningkatan throughput melalui eksekusi konkuren
- Load Distribution: Penyebaran beban komputasi ke multiple cores
- Cache Utilization: Optimalisasi penggunaan CPU cache hierarchy
- Context Switch Minimization: Mengurangi overhead switching antar proses
- System Call Optimization: Efisiensi dalam penggunaan kernel services

KODE C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <sys/time.h>
```

```
#include <time.h>
```

```
#include <string.h>
```

```
#define ARRAY_SIZE 100
```

```
#define NUM_PROCESSES 3
```

```
// Struktur untuk menyimpan hasil proses
```

```
typedef struct {
```

```
    pid_t pid;
```

```
    char task_name[50];
```

```
    double execution_time;
```

```
    int status;
```

```
    char result[100];
```

```
} ProcessResult;
```

```
// Fungsi untuk menghasilkan array acak
```

```
void generate_random_array(int arr[], int size) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        arr[i] = rand() % 1000;
```

```
    }
```

```
}
```

```
// Fungsi untuk mengukur waktu eksekusi
```

```
double get_current_time() {
```



```
struct timeval tv;

gettimeofday(&tv, NULL);

return tv.tv_sec + tv.tv_usec / 1000000.0;

}
```

// Tugas 1: Sorting (Bubble Sort)

```
void task_sorting(int arr[], int size) {

    printf("Process %d (Sorting): Mengurutkan array...\n", getpid());

    for (int i = 0; i < size - 1; i++) {

        for (int j = 0; j < size - i - 1; j++) {

            if (arr[j] > arr[j + 1]) {

                int temp = arr[j];

                arr[j] = arr[j + 1];

                arr[j + 1] = temp;

            }

        }

    }

}
```

// Verifikasi hasil sorting

```
int sorted = 1;

for (int i = 0; i < size - 1; i++) {

    if (arr[i] > arr[i + 1]) {

        sorted = 0;
```

```

        break;
    }
}

printf("Process %d (Sorting): Array %s terurut dengan benar\n",
    getpid(), sorted ? "telah" : "tidak");
}

// Tugas 2: Searching (Linear Search)

void task_searching(int arr[], int size) {

    printf("Process %d (Searching): Mencari nilai dalam array...\n", getpid());

    int target = 42; // Nilai yang dicari

    int found = 0;

    int position = -1;

    for (int i = 0; i < size; i++) {

        if (arr[i] == target) {

            found = 1;

            position = i;

            break;

        }

    }
}

```

```

if (found) {

    printf("Process %d (Searching): Nilai %d ditemukan pada index %d\n",

        getpid(), target, position);

} else {

    printf("Process %d (Searching): Nilai %d tidak ditemukan\n",

        getpid(), target);

}

}

```

// Tugas 3: Calculation (Menghitung statistik)

```

void task_calculation(int arr[], int size) {

    printf("Process %d (Calculation): Menghitung statistik...\n", getpid());

    int sum = 0;

    int min = arr[0];

    int max = arr[0];

    for (int i = 0; i < size; i++) {

        sum += arr[i];

        if (arr[i] < min) min = arr[i];

        if (arr[i] > max) max = arr[i];

    }

    double average = (double)sum / size;

```

```

printf("Process %d (Calculation): Sum=%d, Min=%d, Max=%d, Average=%.2f\n",
      getpid(), sum, min, max, average);
}

```

```

int main() {

    printf("=== SISTEM MANAJEMEN PROSES SEDERHANA ===\n");

    printf("Parent Process PID: %d\n\n", getpid());


    int array[ARRAY_SIZE];

    generate_random_array(array, ARRAY_SIZE);


    pid_t pids[NUM_PROCESSES];

    int pipefd[NUM_PROCESSES][2];

    ProcessResult results[NUM_PROCESSES];


    // Buat pipes untuk komunikasi

    for (int i = 0; i < NUM_PROCESSES; i++) {

        if (pipe(pipefd[i]) == -1) {

            perror("Pipe creation failed");

            exit(1);

        }

    }
}

```

```

// Buat multiple child processes

for (int i = 0; i < NUM_PROCESSES; i++) {

    pids[i] = fork();

    if (pids[i] == -1) {

        perror("Fork failed");

        exit(1);

    }

    if (pids[i] == 0) {

        // Child process

        close(pipefd[i][0]); // Tutup read end

        double start_time = get_current_time();

        // Eksekusi tugas berdasarkan index

        switch (i) {

            case 0:

                task_sorting(array, ARRAY_SIZE);

                strcpy(results[i].task_name, "Sorting Array");

                break;

            case 1:

                task_searching(array, ARRAY_SIZE);

                strcpy(results[i].task_name, "Searching Value");

```

```

        break;

    case 2:

        task_calculation(array, ARRAY_SIZE);

        strcpy(results[i].task_name, "Calculation Stats");

        break;

    }

double end_time = get_current_time();

// Isi data hasil

results[i].pid = getpid();

results[i].execution_time = end_time - start_time;

results[i].status = 0; // Success


// Kirim hasil ke parent melalui pipe

write(pipefd[i][1], &results[i], sizeof(ProcessResult));

close(pipefd[i][1]);


exit(0);

}

}

// Parent process - mengkoordinasi dan mengumpulkan hasil

printf("\n=== PARENT PROCESS MENUNGGU CHILD SELESAI ===\n");

```

```

for (int i = 0; i < NUM_PROCESSES; i++) {

    close(pipefd[i][1]); // Tutup write end di parent


    // Baca hasil dari pipe

    ProcessResult result;

    read(pipefd[i][0], &result, sizeof(ProcessResult));


    results[i] = result;

    close(pipefd[i][0]);


    // Tunggu child process selesai

    int status;

    waitpid(pids[i], &status, 0);


    if (WIFEXITED(status)) {

        results[i].status = WEXITSTATUS(status);

    }

}


// Tampilkan hasil summary

printf("\n=== HASIL EKSEKUSI SEMUA PROSES ===\n");

printf("%-10s %-20s %-12s %-8s\n",

        "PID", "Task", "Time (s)", "Status");

```

```

printf("-----\n");

double total_time = 0;

for (int i = 0; i < NUM_PROCESSES; i++) {

    printf("%-10d %-20s %-12.6f %-8s\n",

        results[i].pid,

        results[i].task_name,

        results[i].execution_time,

        results[i].status == 0 ? "Success" : "Failed");

    total_time += results[i].execution_time;

}

printf("\n=== STATISTIK KESELURUHAN ===\n");

printf("Total processes: %d\n", NUM_PROCESSES);

printf("Total execution time: %.6f seconds\n", total_time);

printf("Parent Process PID: %d\n", getpid());

printf("Semua child processes telah selesai dieksekusi!\n");

return 0;

}

```

2.2 PRAKTIK TUGAS 2

2.2.1 Chat Application dengan IPC

Program chat application ini mengimplementasikan konsep Interprocess Communication (IPC) dengan memanfaatkan shared memory segment sebagai media

penyimpanan pesan terpusat dan semaphore binary sebagai mekanisme sinkronisasi critical section. Arsitektur program menggunakan struktur data chat_room yang berisi array circular buffer untuk menyimpan pesan-pesan dengan format timestamp, username, dan konten pesan, dilengkapi counter untuk manajemen memori dinamis. Mekanisme mutual exclusion dijamin melalui operasi semaphore wait dan signal yang mengontrol akses konkuren ke shared memory selama proses pengiriman dan pembacaan pesan, mencegah race condition yang dapat menyebabkan data corruption. Implementasi ini memungkinkan multiple processes berjalan secara paralel pada terminal berbeda untuk berkomunikasi secara real-time, dengan fitur history yang mengakses seluruh log percakapan secara tersinkronisasi. Program tersebut mendemonstrasikan integrasi efektif antara shared memory management, process synchronization, dan data persistence dalam lingkungan multiprocessing system.

- **Buat file chat.c:**

```
dany@LAPTOP-GRJL2V1D:~$ nano chat.c
dany@LAPTOP-GRJL2V1D:~$ touch chatfile
dany@LAPTOP-GRJL2V1D:~$ gcc chat.c -o chat
```

- **Terminal 1:**

```
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ ./chat User1
=== CHAT ROOM ===
Welcome User1! (Type 'exit' to quit, 'history' to view messages)
User1> Halo Perkenalkan nama Jhodi Andika
User1> Apa hobimu?
User1> history

=== CHAT HISTORY ===
[22:50:45] User1: Halo Perkenalkan nama Jhodi Andika
[22:51:20] User2: Halo Jhodi
[22:51:44] User1: Apa hobimu?
[22:51:53] User2: Main game
=====
User1> exit
Goodbye User1!
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ |
```

- **Terminal 2:**

```
dany@LAPTOP-GRJL2V1D:~$ cd jhodiandika
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ ./chat User2
=== CHAT ROOM ===
Welcome User2! (Type 'exit' to quit, 'history' to view messages)
User2> Halo Jhodi
User2> Main game
User2> exit
Goodbye User2!
dany@LAPTOP-GRJL2V1D:~/jhodiandika$ |
```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

A. PROSES KOMPILASI

- Editing Source Code: Menggunakan nano text editor untuk membuat dan memodifikasi file source code chat.c
- File Preparation: Membuat file chatfile menggunakan command touch sebagai key reference untuk fungsi ftok() dalam IPC
- Kompilasi dengan GCC: Menjalankan gcc chat.c -o chat untuk mengkompilasi program menjadi executable binary
- Preprocessing: Preprocessor mengekspansi semua header files dan makro yang didefinisikan
- Compilation: GCC menerjemahkan kode C menjadi assembly language yang spesifik terhadap arsitektur processor
- Assembly: Assembler mengubah kode assembly menjadi machine code dalam bentuk object file
- Linking: Linker menggabungkan object file dengan system libraries yang diperlukan menghasilkan executable file chat

B. PROSES EKSEKUSI

- Program Initialization: Menjalankan ./chat [username] untuk memulai aplikasi dengan parameter username
- IPC Setup: Membuat shared memory segment dan semaphore menggunakan system calls shmget() dan semget()
- Memory Attachment: Mengattach shared memory ke address space proses current menggunakan shmat()
- Semaphore Initialization: Menginisialisasi semaphore dengan nilai 1 menggunakan semctl() SETVAL operation
- User Interface: Menampilkan welcome message dan command prompt untuk interaksi pengguna

C. MESSAGE PROCESSING

- Input Handling: Menerima input pengguna melalui fgets() dan memproses special commands (exit, history)
- Critical Section Management: Menggunakan semaphore operations (semop dengan SEM_UNDO) untuk mutual exclusion
- Message Formatting: Membuat formatted message dengan timestamp menggunakan localtime() dan strftime()

- Shared Memory Access: Menyimpan pesan ke circular buffer dalam shared memory dengan sinkronisasi penuh
- Circular Buffer Management: Mengimplementasi buffer sirkular untuk handling message overflow dengan shift operation

D. INTERPROCESS COMMUNICATION

- Multi-User Coordination: Multiple instances program dapat berjalan concurrently dengan username berbeda
- Real-Time Synchronization: Semaphore menjamin hanya satu proses yang dapat mengakses shared memory pada satu waktu
- Data Consistency: Mekanisme lock-unlock mencegah race conditions dan memastikan konsistensi data pesan
- History Sharing: Semua proses dapat mengakses complete chat history melalui shared memory yang sama

E. TERMINATION SEQUENCE

- Command Detection: Mendeteksi exit command untuk menginisiasi proses termination
- Resource Cleanup: Melepas shared memory attachment menggunakan shmdt()
- Graceful Exit: Program terminates dengan menampilkan goodbye message dan melepas semua resources
- Persistence: Shared memory tetap aktif untuk processes lain yang masih berjalan

F. KODE C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/sem.h>
```

```
#include <sys/types.h>
```

```
#include <time.h>
```

```
#define SHM_SIZE 1024
```

```
#define MAX_MESSAGES 10
```

```
#define MESSAGE_SIZE 100
```

```
// Struktur untuk shared memory
```

```
typedef struct {
```

```
    char messages[MAX_MESSAGES][MESSAGE_SIZE];
```

```
    int message_count;
```

```
    int read_count;
```

```
} chat_room;
```

```
// Union untuk semctl
```

```
union semun {
```

```
    int val;
```

```
    struct semid_ds *buf;
```

```
    unsigned short *array;
```

```
};
```

```
// Fungsi untuk mengirim pesan
```

```
void send_message(chat_room *room, int sem_id, const char *user, const char *message) {
```

```
    struct sembuf sem_op;
```

```
// Lock semaphore
```

```

sem_op.sem_num = 0;

sem_op.sem_op = -1;

sem_op.sem_flg = 0;

semop(sem_id, &sem_op, 1);


// Format pesan dengan timestamp

time_t now = time(NULL);

struct tm *t = localtime(&now);

char timestamp[20];

snprintf(timestamp, sizeof(timestamp), "%02d:%02d:%02d", t->tm_hour, t->tm_min, t->tm_sec);


// Tambahkan pesan ke chat room

if (room->message_count < MAX_MESSAGES) {

    snprintf(room->messages[room->message_count], MESSAGE_SIZE,

        "[%s] %s: %s", timestamp, user, message);

    room->message_count++;

} else {

    // Geser pesan lama jika sudah penuh

    for (int i = 0; i < MAX_MESSAGES - 1; i++) {

        strcpy(room->messages[i], room->messages[i + 1]);

    }

    snprintf(room->messages[MAX_MESSAGES - 1], MESSAGE_SIZE,

        "[%s] %s: %s", timestamp, user, message);

}

```

```

// Unlock semaphore

sem_op.sem_num = 0;

sem_op.sem_op = 1;

sem_op.sem_flg = 0;

semop(sem_id, &sem_op, 1);
}


// Fungsi untuk menampilkan history chat

void show_history(chat_room *room, int sem_id) {

    struct sembuf sem_op;


// Lock semaphore

sem_op.sem_num = 0;

sem_op.sem_op = -1;

sem_op.sem_flg = 0;

semop(sem_id, &sem_op, 1);


printf("\n=== CHAT HISTORY ===\n");

if (room->message_count == 0) {

    printf("No messages yet.\n");

} else {

    for (int i = 0; i < room->message_count; i++) {

        printf("%s\n", room->messages[i]);

```

```

    }

}

printf("=====\n");

// Unlock semaphore

sem_op.sem_num = 0;

sem_op.sem_op = 1;

sem_op.sem_flg = 0;

semop(sem_id, &sem_op, 1);

}

int main(int argc, char *argv[]) {

    if (argc != 2) {

        printf("Usage: %s <username>\n", argv[0]);

        printf("Example: %s User1\n", argv[0]);

        return 1;

    }

    char *username = argv[1];

    key_t key = ftok("chatfile", 65);

    int shm_id, sem_id;

    chat_room *room;

    union semun sem_arg;

```

```
// Buat shared memory

shm_id = shmget(key, sizeof(chat_room), 0666 | IPC_CREAT);

if (shm_id == -1) {

    perror("shmget failed");

    return 1;

}
```

```
// Attach shared memory

room = (chat_room*) shmat(shm_id, NULL, 0);

if (room == (void*) -1) {

    perror("shmat failed");

    return 1;

}
```

```
// Inisialisasi shared memory jika pertama kali

if (shmctl(shm_id, IPC_STAT, NULL) == 0) {

    room->message_count = 0;

    room->read_count = 0;

}
```

```
// Buat semaphore

sem_id = semget(key, 1, 0666 | IPC_CREAT);

if (sem_id == -1) {

    perror("semget failed");

}
```



```

    return 1;
}

// Inisialisasi semaphore
sem_arg.val = 1;
if (semctl(sem_id, 0, SETVAL, sem_arg) == -1) {
    perror("semctl failed");
    return 1;
}

printf("=== CHAT ROOM ===\n");
printf("Welcome %s! (Type 'exit' to quit, 'history' to view messages)\n", username);

char message[MESSAGE_SIZE];
while (1) {
    printf("%s> ", username);
    fflush(stdout);

    if (fgets(message, sizeof(message), stdin) == NULL) {
        break;
    }

    // Hapus newline
    message[strcspn(message, "\n")] = 0;

```

```
// Periksa perintah

if (strcmp(message, "exit") == 0) {

    break;

} else if (strcmp(message, "history") == 0) {

    show_history(room, sem_id);

} else if (strlen(message) > 0) {

    send_message(room, sem_id, username, message);

}

}

// Detach shared memory

shmdt(room);

printf("Goodbye %s!\n", username);

return 0;

}
```

BAB III

PENUTUP

3.1 KESIMPULAN

Berdasarkan pelaksanaan dan evaluasi komprehensif yang telah dilakukan, dapat disimpulkan bahwa eksperimen sistem operasi ini berhasil mengilustrasikan prinsip-prinsip inti manajemen proses dan Komunikasi Antar Proses (IPC) secara mendalam. Dalam modul manajemen proses, pola fork-exec-wait menunjukkan efektivitasnya dalam membentuk struktur hierarkis proses, dimana proses induk berfungsi sebagai pengendali yang mengkoordinir berbagai proses anak dengan tugas terspesialisasi, sementara pipa berperan sebagai saluran komunikasi satu arah yang andal untuk mengonsolidasi hasil komputasi.

Pada pengembangan aplikasi chat, segmen memori bersama (shared memory) berhasil diimplementasikan sebagai repositori pesan terpusat yang hemat sumber daya dengan kinerja tinggi, didukung oleh semaphore biner yang menjamin eksklusivitas mutual pada bagian kritis sehingga secara efektif mencegah kondisi race. Penerapan circular buffer dalam arsitektur memori bersama memungkinkan pengelolaan memori yang adaptif dan stabil meskipun menghadapi multiple akses bersamaan.

Kedua implementasi ini membuktikan bahwa pemahaman mendalam tentang sinkronisasi proses dan mekanisme IPC merupakan prasyarat fundamental dalam pengembangan sistem terdistribusi dan aplikasi konkuren yang membutuhkan optimalisasi berbagi sumber daya dan konsistensi data. Temuan praktikum ini tidak hanya memberikan landasan kokoh untuk pengembangan sistem operasi yang lebih kompleks, tetapi juga membuka jalan bagi terciptanya aplikasi real-time dengan persyaratan ketersediaan tinggi dan ketahanan terhadap kegagalan.

Hasil eksperimen ini secara tegas menegaskan bahwa penguasaan konsep manajemen proses dan IPC merupakan kompetensi krusial dalam merancang solusi komputasi modern yang mampu memenuhi tuntutan skalabilitas, keandalan, dan efisiensi dalam lingkungan komputasi terkini.